CONFIDENTIAL

UNIVERSITI TEKNOLOGI MALAYSIA
FACULTY OF COMPUTING

TEST 2
SEMESTER I 2023/2024

SUBJECT CODE	**: SECJ2013**
SUBJECT NAME	**: DATA STRUCTURES AND ALGORITHMS**
YEAR/COURSE	**: 2 (SECJH/ SECVH/ SECBH/ SECRH/ SECPH)**
TIME	**: 8.00 PM – 10.00 PM**
DATE	**: 3rd JANUARY 2024 (WEDNESDAY)**
VENUE	**: N28 MPK1-MPK10, CGMTL, CCNA, CSL, MCASE, MJIIT**

INSTRUCTIONS TO THE STUDENTS:

- This is a **CLOSED-BOOK** practical test. The test will require you to code on a computer.
- References to any resources, including any source code, websites, etc., are strictly prohibited.
- You should code **OFFLINE** using any C++ IDE such as VS Code, DevCPP, etc.
- The use of any form of artificial intelligence (AI) tool is strictly prohibited. Such tools include ChatGPT, AI-based VS Code extensions like Co-Pilot, etc.
- Be warned that your program will undergo a screening process for plagiarism and AI detection.
- You must use only the computers in the lab. You are not allowed to use your own laptop to attempt this test.
- All comments in the program will not be graded.
- Programs that cannot compile will receive a penalty.
- Any question related to the test question will not be entertained at all. Please read the question carefully.

IMPORTANT NOTES:

- **Starter** codes are provided for this test.
- You need to write the program **in the starter code**.
- Write each program using **inline style** and use only a **single source code file**.

This question paper consists of FIVE (5) printed pages including this page.

Question 1 [50 Marks]

Given a C++ program file, **Test2_Link.cpp** that implement a **circular linked-list**. You are provided with two classes, **Node** and **List**.

1. The **Node** class is **fully** provided (refer to **Figure 1**). This class represents a node in the circular linked-list. Each node contains a character and a link to the next node in the list.

```
class Node {
  char ch;

  public:
    Node* next;
    Node(char c){ ch = c; next = NULL; }
    char getChar() { return ch; }
};
```

Figure 1: Complete Node class definition

2. The **List** class is **partially** provided (refer to **Figure 2**). This class represents the circular linked-list itself.

```
class List {
  Node* head;

  public:
    List() { head = NULL; }
    bool isEmpty() { return head == NULL; }

    void insertNode(Node *);
    void deleteNode();
    string findNode(char);
    void displayList();
};
```

Figure 2: Incomplete List class definition

You are required to complete several methods in this class based on the descriptions given below:

a) **insertNode()** - This method should insert a new node at the **beginning** of the list. You need to consider two cases: (9.5 marks)

   i) When the list is empty.

   ii) When the list already has nodes.

b) **deleteNode()** - This method should delete the node at the **end** of the list. You need to consider three cases: (12 marks)

i) When the list is empty, display "*No nodes to delete.*" message.

ii) When the list has multiple nodes.

iii) When the list only has one node.

c) **findNode()** - This method should return the word **"True"** if the node being searched for is in the list, and return the word **"False"** if the node being searched for is not in the list.

(8 marks)

d) **displayList()** - This method should display all nodes in the list. However, if the list is empty, it should display "*Nothing to display.*" message. (7.5 marks)

3. In the **main()** method: (13 marks)

a) Create a list.

b) Using the list created in 3(a), you need to call the appropriate method(s) created in 2 to produce the output as shown in **Figure 3**.

```
No nodes to delete.
A
Z -> A
Z is found in the list? True
Z
Nothing to display.
```

Figure 3: Output of node operations in the completed program

Question 2 [50 Marks]

Given a C++ program file, **Test2_Stack.cpp** with array implementation of **stack**. The implementation of the method `isEmpty(), isFull(), push()`, and `pop()` are given in the program. Based on the description below, complete the task of the implementation for `stackTop()`, `display()` and `checkSize()` methods. Then, using all of the defined methods, complete the `main()` function by appending the appropriate line of code at the space provided to generate the output shown in Figure 2(a), (b) and (c). You must **NOT change** any line of code regarding the definition of the given method.

Task 1 :
Complete the implementation of the `stackTop()` method to get the top element in the stack. (8 marks)

Task 2 :
Complete the implementation of the `display()` method to display all of the elements in the stack starting at the top. (12 marks)

Task 3 :
Complete the implementation of the `checkSize()` method to get the number of elements in the stack. (8 marks)

Task 4 :
Complete the **main** method with a few calls and/or statements. Use the methods defined in the program to perform the following operations:

i. Use the appropriate code that able to push `theCars` elements into the stack. Display all the elements in the stack as shown in Figure 2(a) (8 marks)

```
Vios
Shilpy
Nissan
Mazda
Velphire
Alphard
Estima
```

Figure 2(a): After Task 4(i) is completed

ii. Use the appropriate code that able to able to remove and display the removed 2 top elements in the stack as shown in Figure 2(b) (6 marks)

```
Remove Vios
Remove Shilpy
```

Figure 2(b): After Task 4(ii) is completed

iii. Display all the elements in the stack. Use the appropriate code that able to display the number of elements in the stack as shown in Figure 2(c) (8 marks)

```
Nissan
Mazda
Velphire
Alphard
Estima

#Cars in ShowRoom : 5
```

Figure 2(c): after Task 4(iii) is completed